

LOGISTIC REGRESSION

Concise Study Notes for Exam & Interview Preparation

Sigmoid Function | Binary Cross-Entropy | Gradient Descent | Regularization | Practice Questions

Contents

1	Introduction to Logistic Regression
2	Why Not Linear Regression for Classification?
3	The Sigmoid Function & Decision Boundary
4	Mathematical Formulation
5	Cost Function: Binary Cross-Entropy
6	Gradient Descent & Learning the Parameters
7	Regularization (L1 & L2)
8	Multiclass Logistic Regression
9	Assumptions & Evaluation Metrics
10	Advantages & Disadvantages
11	Real-World Applications
12	Formula Cheat-Sheet
13	Practice Questions



Introduction to Logistic Regression

Logistic Regression is a supervised machine learning algorithm used for **classification**, most commonly binary classification (two classes, labelled 0 and 1). Despite the name "regression", it is a classification algorithm: instead of predicting a continuous value, it predicts the **probability** that an input belongs to a particular class.

It works by taking a linear combination of input features, then passing that result through the **sigmoid function** to squeeze the output into the range $[0, 1]$ — a valid probability. A threshold (typically 0.5) is then used to convert this probability into a class label.

Why Logistic Regression matters

It is one of the simplest, fastest, and most interpretable classification algorithms, and is often the first baseline model tried on a new classification problem. It also forms the mathematical foundation for the output layer of neural network classifiers.

Key terms you must know:

Term	Meaning
Sigmoid / Logistic function	An S-shaped function that maps any real number into the range $(0, 1)$.
Decision boundary	The surface (line, in 2D) where the predicted probability equals 0.5; separates predicted classes.
Log-odds (logit)	The natural logarithm of the odds ratio, $p/(1-p)$; logistic regression models this as a linear function of the inputs.
Binary cross-entropy	The loss function used to measure how well predicted probabilities match true labels.



Why Not Linear Regression for Classification?

It is tempting to use ordinary linear regression for a 0/1 classification problem — just fit a straight line and threshold it at 0.5. This approach has serious problems:

- **Unbounded output.** A linear model $w \cdot x + b$ can output any real number, including values far below 0 or above 1 — meaningless as a probability.
- **Sensitive to outliers.** A single extreme data point can drag the regression line and shift the decision threshold for every other point.
- **Wrong error structure.** Linear regression assumes continuous, normally-distributed errors, which does not match binary-valued labels.
- **Poor fit shape.** A straight line cannot capture the natural "S-shaped" transition between class probabilities seen in real data.

Logistic regression solves all four issues by passing the linear combination through the sigmoid function, which naturally bounds output between 0 and 1 and produces the characteristic S-curve.



The Sigmoid Function & Decision Boundary

3.1 The Sigmoid Function

The sigmoid (or logistic) function $\sigma(z)$ converts any real-valued input z into a number between 0 and 1:

$$\sigma(z) = 1 / (1 + e^{-z})$$

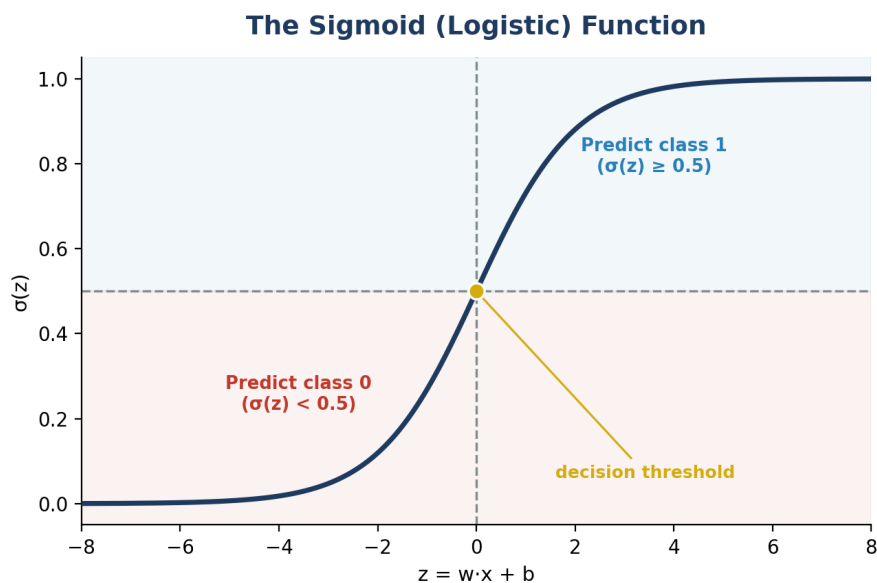


Figure 3.1 — The sigmoid function smoothly maps any real number to a probability between 0 and 1, with $\sigma(0) = 0.5$ as the natural decision threshold.

3.2 Properties

- $\sigma(z) \rightarrow 1$ as $z \rightarrow +\infty$, and $\sigma(z) \rightarrow 0$ as $z \rightarrow -\infty$.
- $\sigma(0) = 0.5$ exactly, which is why 0.5 is the default classification threshold.
- The function is monotonically increasing and differentiable everywhere — important for gradient-based optimization.
- Its derivative has the elegant closed form: $\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$.

Mathematical Formulation

4.1 Hypothesis Function

Given input features x , weights w , and bias b , logistic regression first computes a linear score z , then squashes it with the sigmoid:

$$z = w \cdot x + b \quad h(x) = \sigma(z) = P(y = 1 \mid x)$$

$h(x)$ is interpreted directly as the model's estimated probability that the input belongs to class 1. The predicted class is then:

$$y_{\text{pred}} = 1 \text{ if } h(x) \geq 0.5, \text{ else } y_{\text{pred}} = 0$$

4.2 Decision Boundary in Feature Space

Because $\sigma(z) = 0.5$ exactly when $z = 0$, the decision boundary is simply the set of points where $w \cdot x + b = 0$ — a straight line in 2D, a plane in 3D, and a hyperplane in general. Everything on one side is classified as 1, everything on the other as 0.

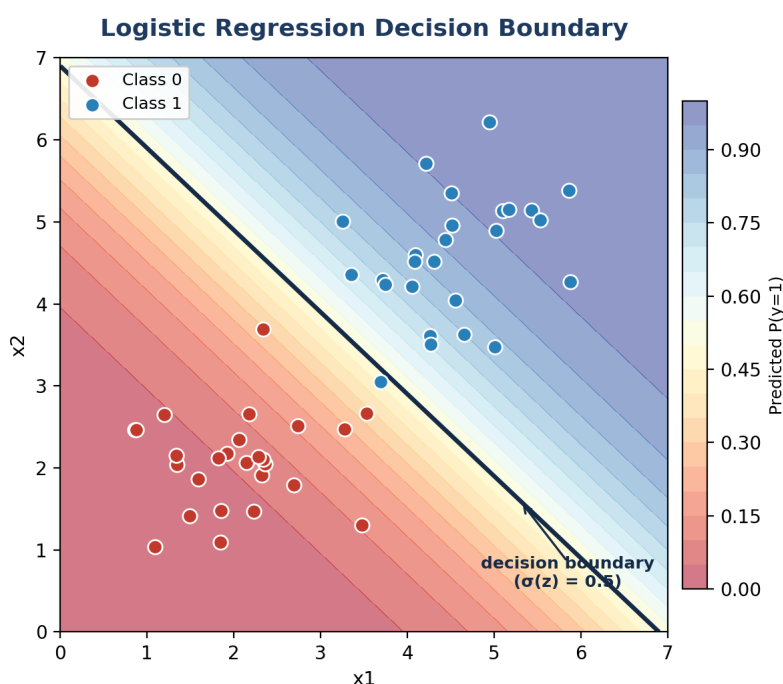


Figure 4.1 — The model assigns a smooth probability gradient across feature space; the dark line marks where the predicted probability is exactly 0.5.

4.3 Log-Odds (Logit) Interpretation

Rearranging $h(x) = \sigma(z)$ shows that logistic regression models the **log-odds** of the positive class as a linear function of the inputs:

$$\log\left(\frac{h(x)}{1 - h(x)}\right) = w \cdot x + b$$

This is why each coefficient w_j has a clean interpretation: a one-unit increase in feature x_j changes the log-odds of $y=1$ by w_j , holding other features constant.



Cost Function: Binary Cross-Entropy

If we tried to use mean squared error (as in linear regression) with the sigmoid hypothesis, the resulting cost surface would be **non-convex**, riddled with local minima — gradient descent could get stuck. Instead, logistic regression uses **binary cross-entropy** (log loss), which is convex for this model.

5.1 Per-Example Cost

$$\text{Cost}(h(x), y) = -\log(h(x)) \quad \text{if } y = 1$$

$$\text{Cost}(h(x), y) = -\log(1 - h(x)) \quad \text{if } y = 0$$

Log-Loss Heavily Penalizes Confident, Wrong Predictions

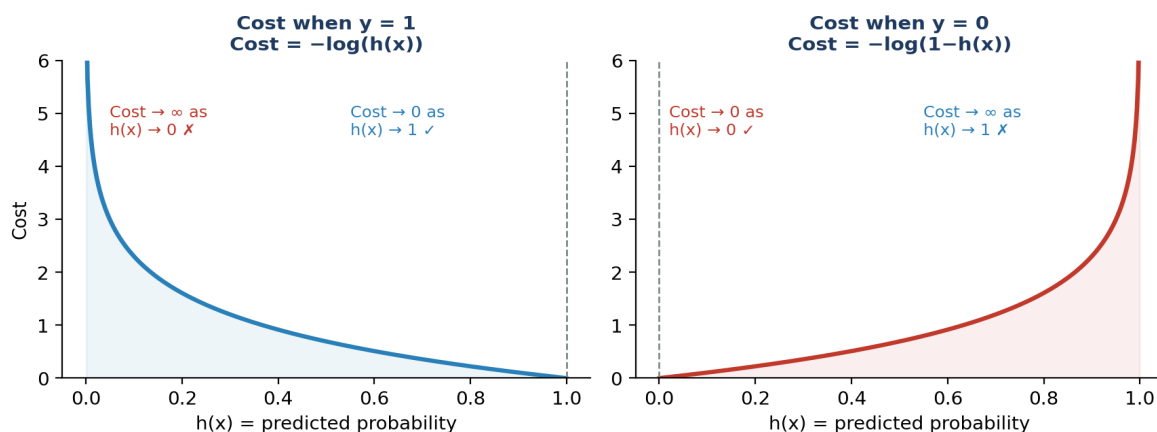


Figure 5.1 — Log-loss is near zero for confident, correct predictions, but grows without bound as a confident prediction turns out to be wrong.

5.2 Combined Formula & Overall Cost

Both cases can be written in a single expression (since y is either 0 or 1, only one term is active at a time):

$$\text{Cost}(h(x), y) = -[y \cdot \log(h(x)) + (1-y) \cdot \log(1-h(x))]$$

Averaging this over all m training examples gives the full cost function $J(w, b)$ that training seeks to minimize:

$$J(w, b) = -(1/m) \sum [y_i \cdot \log(h(x_i)) + (1-y_i) \cdot \log(1-h(x_i))]$$



Gradient Descent & Learning the Parameters

Since $J(w, b)$ is convex for logistic regression, gradient descent reliably converges to the global minimum. The parameters are updated iteratively in the direction that most reduces the cost.

6.1 Gradient of the Cost Function

Remarkably, despite the more complex log-loss formula, the gradient of J with respect to each weight has exactly the same elegant form as in linear regression:

$$\partial J / \partial w_j = (1/m) \sum (h(x_i) - y_i) \cdot x_{ij}$$

6.2 Update Rule

$$w_j := w_j - \alpha \cdot \partial J / \partial w_j \quad (\text{repeat until convergence})$$

Here α is the **learning rate**. All weights (and the bias) are updated simultaneously on every iteration. In practice, variants such as **stochastic gradient descent** or **mini-batch gradient descent** are used for large datasets, and optimizers like Newton's method or L-BFGS are common alternatives in statistical software.

Note

Unlike linear regression, there is no closed-form solution for logistic regression's weights — the cost function must be minimized iteratively.



Regularization (L1 & L2)

With many features (or a small training set), logistic regression can overfit — learning weights that fit noise in the training data. **Regularization** discourages large weights by adding a penalty term to the cost function.

Type	Penalty added to J	Effect
L2 (Ridge)	$(\lambda/2m) \sum w_j^2$	Shrinks all weights smoothly toward zero; rarely sets any weight exactly to zero
L1 (Lasso)	$(\lambda/m) \sum w_j $	Can shrink some weights exactly to zero, performing automatic feature selection
Elastic Net	Weighted mix of L1 and L2	Balances feature selection (L1) with stability (L2)

λ (lambda) is the regularization strength: $\lambda = 0$ means no regularization (the original cost function), while a very large λ forces weights toward zero, simplifying the model but risking underfitting. Note that the bias term b is conventionally left out of the penalty.



Multiclass Logistic Regression

Logistic regression is inherently binary, but two standard strategies extend it to problems with more than two classes.

8.1 One-vs-Rest (OvR)

Train one binary classifier per class, each distinguishing "this class" versus "all other classes". At prediction time, run all classifiers and pick the class with the highest predicted probability. Simple and works with any binary classifier, but the K classifiers are trained independently.

8.2 Softmax (Multinomial) Regression

A direct generalization that jointly outputs a full probability distribution over K classes using the softmax function:

$$P(y = k | x) = e^{z_k} / \sum_j e^{z_j}$$

where $z_k = w_k \cdot x + b_k$ is the linear score for class k. The probabilities across all K classes sum to exactly 1, and the model is trained jointly with a multiclass cross-entropy loss — generally preferred over OvR when the underlying library supports it directly.



Assumptions & Evaluation Metrics

9.1 Key Assumptions

- The outcome variable is categorical (binary for standard logistic regression).
- Observations are independent of each other.
- Little or no multicollinearity among the independent variables.
- The log-odds of the outcome is a linear function of the input features.
- Large sample size is preferred for stable, reliable coefficient estimates.

9.2 Common Evaluation Metrics

Metric	What it measures
Accuracy	Fraction of all predictions that are correct; misleading on imbalanced data
Precision	Of all predicted positives, how many were actually positive: $TP/(TP+FP)$
Recall (Sensitivity)	Of all actual positives, how many were correctly found: $TP/(TP+FN)$
F1-Score	Harmonic mean of precision and recall; balances both concerns
ROC-AUC	Area under the ROC curve; measures ranking quality across all thresholds

Practical Tip

Always check class balance before trusting accuracy alone. On an imbalanced dataset (e.g. 95% negative class), a model that always predicts the majority class achieves 95% accuracy while being completely useless — precision, recall, and ROC-AUC tell a much more honest story.

Advantages & Disadvantages

Advantages	Disadvantages
Simple, fast to train, and easy to implement	Assumes a linear decision boundary in log-odds space; struggles with complex non-linear patterns
Highly interpretable — coefficients show each feature's effect on log-odds	Sensitive to multicollinearity among input features
Outputs well-calibrated probabilities, not just hard labels	Requires reasonably large sample sizes for stable estimates
Works well when classes are (approximately) linearly separable	Can underperform more flexible models (trees, SVM, neural nets) on complex data
Naturally extends to multiclass problems via softmax	Vulnerable to outliers when classes are not well separated



Real-World Applications

Medical diagnosis. Predicting probability of disease presence (e.g. diabetes, heart disease) from patient measurements.

Credit scoring. Estimating the probability that a loan applicant will default, based on financial history.

Marketing & churn prediction. Predicting whether a customer will click an ad, convert, or cancel a subscription.

Spam detection. Classifying emails as spam or not-spam based on text and metadata features.

A/B testing analysis. Modelling the probability of a binary outcome (e.g. conversion) as a function of experimental treatment and covariates.

Formula Cheat-Sheet

Concept	Formula
Sigmoid function	$\sigma(z) = 1 / (1 + e^{-z})$
Hypothesis	$h(x) = \sigma(w \cdot x + b)$
Decision rule	$y_{\text{pred}} = 1 \text{ if } h(x) \geq 0.5, \text{ else } 0$
Log-odds (logit)	$\log(h(x)/(1-h(x))) = w \cdot x + b$
Per-example cost	$-[y \cdot \log(h(x)) + (1-y) \cdot \log(1-h(x))]$
Overall cost $J(w,b)$	$-(1/m) \sum [y_i \cdot \log(h(x_i)) + (1-y_i) \cdot \log(1-h(x_i))]$
Gradient	$\partial J / \partial w_j = (1/m) \sum (h(x_i) - y_i) \cdot x_{ij}$
Update rule	$w_j := w_j - \alpha \cdot \partial J / \partial w_j$
L2 penalty	$(\lambda/2m) \sum w_j^2$
Softmax (multiclass)	$P(y=k x) = e^{z_k} / \sum_j e^{z_j}$



Practice Questions

Short-answer and conceptual questions — try answering before checking the hints.

Q1. Why can't we just use linear regression with a 0.5 threshold for classification?

Hint: think about unbounded outputs, outlier sensitivity, and the shape of real class-probability transitions.

Q2. Derive the decision boundary equation from the condition $h(x) = 0.5$.

Hint: solve $\sigma(z) = 0.5$ for z , then substitute $z = w \cdot x + b$.

Q3. Why is binary cross-entropy used instead of mean squared error as the cost function?

Hint: think about convexity of the resulting cost surface with a sigmoid hypothesis.

Q4. What does a positive coefficient w_j mean in terms of log-odds?

Hint: recall the log-odds equation — a one-unit increase in x_j changes log-odds by exactly w_j .

Q5. Why does logistic regression have no closed-form solution like linear regression?

Hint: think about the non-linearity introduced by the sigmoid inside the log-loss function.

Q6. Explain the difference between L1 and L2 regularization in terms of their effect on weights.

Hint: L1 can drive weights to exactly zero; L2 only shrinks them smoothly.

Q7. Why might accuracy be a misleading metric for an imbalanced classification problem?

Hint: consider what happens if a model just predicts the majority class every time.

Q8. Compare One-vs-Rest and Softmax regression for multiclass problems.

Hint: think about whether classifiers are trained jointly and whether outputs form a valid probability distribution.

Q9. What is the derivative of the sigmoid function, and why is it useful?

Hint: $\sigma'(z) = \sigma(z)(1-\sigma(z))$ — this clean form simplifies the gradient computation.

Q10. List two real-world scenarios where you would prefer probability outputs over hard class labels.

Hint: think about medical risk scoring or credit risk, where the degree of risk matters, not just yes/no.

Study Tip

Before an exam, make sure you can: (1) write the sigmoid function and sketch its S-curve, (2) write the log-loss cost function and explain why it's convex, (3) state the gradient descent update rule, and (4) explain L1 vs L2 regularization. These four cover the large majority of typical logistic regression exam questions.